Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования

Национальный исследовательский Нижегородский государственный университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

ОТЧЁТ

по лабораторной работе № 1

«Множества на основе битовых полей»

Выполнил:

студент группы 3825Б1Фисп1

Титов А.И.

Проверил:

Ивлев А.Д.

Нижний Новгород

2026

Содержание

Введение

1. Постановка задачи

2. Руководство пользователя

3. Руководство программиста

3.1. Описание структуры программы

3.2. Описание структур данных

3.3. Описание алгоритмов

4. Результаты экспериментов и тестирования

4.1. Анализ покрытия тестами

Заключение

Литература

Приложение А. Формулы и основные операции

Приложение Б. Сведения о тестировании

Введение

Лабораторная работа посвящена представлению конечных множеств с помощью битовых полей. Если универсум заранее известен и его элементы пронумерованы от 0 до N−1, принадлежность каждого элемента можно хранить одним битом: 1 означает, что элемент входит в множество, 0 — что не входит.

Реализованы два класса C++. TBitField отвечает за хранение и обработку последовательности битов. TSet использует объект TBitField как характеристический вектор множества. Такое разделение позволяет отделить низкоуровневую работу с памятью от теоретико-множественных операций.

Основное внимание уделено структурам данных, алгоритмам и оценкам сложности. Для методов приводятся параметры, результат работы и асимптотическая оценка. Дополнительно разработаны автоматические тесты Google Test, проверяющие обычные сценарии и граничные случаи.

1. Постановка задачи

Цель работы — разработать структуру данных для хранения множеств с использованием битовых полей и освоить фреймворк Google Test.

Требовалось: реализовать класс TBitField согласно заданному интерфейсу; реализовать класс TSet согласно заданному интерфейсу; написать тесты, покрывающие основной функционал и краевые случаи.

Объявления классов в предоставленных заголовочных файлах должны оставаться неизменными. Реализация методов размещена в отдельных файлах tbitfield.cpp и tset.cpp. Тесты размещены в test.cpp.

Универсум множества имеет мощность MaxPower, элементы нумеруются от 0 до MaxPower−1. Объединение и пересечение двух множеств выполняются при совпадении мощностей универсумов. В реализованной версии при несовпадении размеров возвращается пустое множество.

Отрицательная длина TBitField и отрицательная мощность TSet нормализуются к нулю. Обращение по недопустимому индексу не должно приводить к доступу за пределы массива.

2. Руководство пользователя

Реализация является набором классов и автоматических тестов, а не самостоятельным приложением с меню. Для использования классов необходимо подключить tbitfield.h и tset.h, а также добавить в проект tbitfield.cpp и tset.cpp.

Для создания битового поля используется TBitField field(8). При создании все биты равны нулю. SetBit(i) устанавливает бит, ClrBit(i) очищает его, GetBit(i) возвращает его значение.

Для создания множества используется TSet set(8). Допустимые элементы имеют номера 0–7. InsElem(i) добавляет элемент, DelElem(i) удаляет, IsMember(i) возвращает 1, если элемент входит в множество, и 0 в противном случае.

Операторы + и - с целым аргументом возвращают копию множества с добавленным или удалённым элементом. Оператор + для двух множеств выполняет объединение, * — пересечение, ~ — дополнение относительно универсума. Операторы == и != сравнивают множества.

Потоковый вывод TBitField отображает последовательность битов. TSet выводится в фигурных скобках, например {1,3,6}. Ввод принимает последовательность 0 и 1. Значение, отличное от 0 и 1, вызывает установку std::ios::failbit.

Для проверки необходимо собрать тестовый проект Visual Studio и запустить тесты через Test Explorer или тестовое исполняемое приложение. Успешное выполнение тестов отображается как Passed.

3. Руководство программиста. 3.1. Структура программы

Программа разделена на заголовочные файлы с интерфейсами классов, cpp-файлы с реализациями и файл автоматических тестов.

tbitfield.h объявляет тип TELEM как unsigned int и класс TBitField. В классе находятся поля BitLen, pMem и MemLen, вспомогательные методы GetMemIndex и GetMemMask, конструкторы, деструктор, методы доступа к битам, операторы сравнения, присваивания и побитовые операции.

tbitfield.cpp реализует выделение и освобождение памяти, вычисление индекса и маски, изменение и чтение битов, копирование и побитовые операции.

tset.h объявляет класс TSet. Он содержит MaxPower и вложенный объект BitField типа TBitField. Интерфейс включает конструкторы, преобразование к TBitField, операции включения и удаления элементов, проверку принадлежности, сравнение, объединение, пересечение и дополнение.

tset.cpp реализует методы TSet через операции вложенного BitField. test.cpp содержит тесты Google Test, разделённые на группы TBitFieldTest и TSetTest. Глобальное хранилище данных не используется.

3.2. Структуры данных. Класс TBitField

TBitField хранит последовательность битов фиксированной длины. Поле BitLen содержит число логически доступных битов. pMem — указатель на динамический массив элементов TELEM. MemLen — количество элементов массива, необходимое для хранения BitLen битов.

TELEM определён как unsigned int. Количество битов в одном элементе вычисляется как bitsElement = sizeof(TELEM) * 8. Для N логических битов требуется MemLen = (N + bitsElement − 1) / bitsElement элементов. Это целочисленное округление вверх.

Биты внутри каждого TELEM нумеруются от младшего к старшему. Для бита n индекс элемента массива равен n / bitsElement, а позиция внутри элемента — n % bitsElement.

GetMemIndex возвращает индекс элемента массива. GetMemMask создаёт маску, содержащую единицу на позиции n % bitsElement: TELEM(1) << (n % bitsElement).

Такое представление компактно: один элемент TELEM хранит сразу несколько логических значений. При нулевой длине память не выделяется, pMem равен nullptr. При положительной длине массив выделяется с нулевой инициализацией.

3.2. Структуры данных. Класс TSet

TSet представляет конечное множество на универсуме U={0,1,...,N−1}. В качестве представления используется характеристический вектор: бит i равен 1, если элемент i входит в множество, и 0 в противном случае.

MaxPower хранит мощность универсума. BitField хранит характеристический вектор длины MaxPower. Эти поля согласованы конструкторами класса.

Конструктор TSet(int mp) создаёт множество заданной мощности; отрицательное значение заменяется нулём. Конструктор копирования создаёт независимую копию. Конструктор из TBitField устанавливает MaxPower равным длине переданного поля.

Оператор преобразования TSet в TBitField возвращает копию характеристического вектора. Сам класс TSet не работает с массивом TELEM напрямую, а делегирует операции классу TBitField. Это уменьшает дублирование кода.

Номер допустимого элемента совпадает с номером бита в характеристическом векторе. Поэтому включение, удаление и проверка принадлежности выполняются через SetBit, ClrBit и GetBit.

3.3. Алгоритмы. Доступ к биту

Перед обращением к биту проверяется диапазон. Если n<0 или n>=BitLen, SetBit и ClrBit завершаются без изменения данных, а GetBit возвращает 0.

Для допустимого номера вычисляются индекс слова index=n/W и маска mask=1u<<(n%W), где W — количество битов в TELEM.

Установка выполняется выражением pMem[index] |= mask. Побитовое ИЛИ с маской устанавливает выбранный бит в 1, не меняя остальные.

Очистка выполняется pMem[index] &= ~mask. Инвертированная маска содержит ноль на выбранной позиции и единицы на остальных, поэтому очищается только нужный бит.

Чтение выполняется проверкой (pMem[index] & mask) != 0. Если результат ненулевой, возвращается 1, иначе 0.

Вычисление индекса и маски требует постоянного числа операций, не зависящего от длины поля. SetBit, ClrBit и GetBit имеют временную сложность O(1), дополнительная память — O(1).

3.3. Алгоритмы. Создание, копирование и сравнение

Конструктор TBitField нормализует отрицательную длину, вычисляет BitLen и MemLen, затем выделяет массив TELEM с нулевой инициализацией. Количество слов пропорционально N, поэтому время и память конструктора оцениваются как O(N).

Конструктор копирования выделяет новый массив и копирует каждый элемент массива источника. Копия владеет независимой памятью. Время работы и дополнительная память — O(N).

Оператор присваивания проверяет самоприсваивание. Затем выделяет новый массив и копирует данные источника. Только после успешного копирования освобождается прежняя память объекта и обновляются поля. Такой порядок сохраняет старое состояние при неудачном выделении памяти. Сложность — O(N).

Оператор == сначала сравнивает длины. При различии длины поля не равны. При совпадении последовательно сравниваются элементы pMem; при первом различии возвращается 0. В худшем случае просматривается весь массив — O(N).

Оператор != возвращает отрицание результата ==, поэтому имеет ту же сложность O(N). GetLength возвращает BitLen без обхода массива и выполняется за O(1).

3.3. Алгоритмы. Побитовые операции

Оператор | выполняет побитовое ИЛИ. Сначала проверяется равенство BitLen двух операндов. Если длины различаются, возвращается TBitField(0). Иначе создаётся результат той же длины и для каждого машинного слова вычисляется pMem[i] | bf.pMem[i]. Сложность — O(N).

Оператор & работает аналогично, но использует побитовое И. Бит результата равен 1 только тогда, когда он равен 1 в обоих операндах. При несовпадении длин возвращается пустое поле. Сложность — O(N).

Оператор ~ создаёт поле той же длины и перебирает все логические позиции. Если исходный бит равен 0, в результате устанавливается 1. Если исходный бит равен 1, соответствующий бит результата остаётся нулём.

Дополнение ограничено логической длиной BitLen: неиспользуемые старшие позиции последнего машинного слова не становятся частью результата. Временная сложность — O(N), память результата — O(N).

Операции | и & не расширяют меньший операнд и не вычисляют общую часть при разных длинах. Возврат пустого поля — выбранное поведение реализации, отдельно проверяемое тестами.

3.3. Алгоритмы. Операции над множествами

InsElem проверяет, что Elem находится в диапазоне [0,MaxPower). При допустимом значении вызывает BitField.SetBit(Elem). DelElem выполняет такую же проверку и вызывает ClrBit. IsMember возвращает 0 для недопустимого номера, иначе вызывает GetBit. Каждая из этих операций имеет сложность O(1).

Объединение оператором + сначала проверяет равенство MaxPower. При несовпадении возвращается TSet(0). Иначе создаётся результат той же мощности, а его битовое поле получает результат побитового ИЛИ. Сложность — O(N).

Пересечение оператором * выполняется аналогично, но используется побитовое И. В результате остаются только элементы, присутствующие в обоих множествах. Сложность — O(N).

Дополнение оператором ~ создаёт множество той же мощности и присваивает его BitField результат операции ~ над исходным полем. Элементы, отсутствующие в исходном множестве, входят в результат. Сложность — O(N).

Операторы + и - с целым аргументом сначала копируют исходное множество, затем изменяют копию через InsElem или DelElem. Исходный объект не меняется. Из-за копирования сложность этих операторов O(N), хотя само изменение одного элемента занимает O(1).

Сравнение множеств проверяет MaxPower, а затем сравнивает характеристические векторы. В худшем случае сложность O(N).

3.3. Алгоритмы. Ввод и вывод

Вывод TBitField перебирает позиции от 0 до BitLen−1 и печатает каждый бит как 0 или 1. Для N битов выполняется N итераций, сложность O(N).

Ввод TBitField считывает BitLen целых значений. При значении 1 бит устанавливается, при 0 очищается. Если чтение не удалось, оператор возвращает поток. Если значение не равно 0 или 1, устанавливается std::ios::failbit и ввод прекращается.

Если ошибка возникает посередине ввода, уже обработанные биты остаются изменёнными, а ещё не обработанные сохраняют прежние значения. Это следует из последовательного характера алгоритма.

Ввод TSet аналогично считывает MaxPower значений и для каждого номера включает или удаляет соответствующий элемент. Значение вне {0,1} приводит к failbit.

Вывод TSet формирует строку в фигурных скобках и перечисляет присутствующие элементы через запятую. Пустое множество отображается как {}. Ввод и вывод обрабатывают N элементов, поэтому имеют временную сложность O(N), дополнительная память — O(1).

3.3. Сводная оценка сложности

Обозначим через N длину битового поля или мощность универсума, а через W — число битов в TELEM. W является фиксированной характеристикой платформы.

К O(1) относятся GetLength, GetMaxPower, GetMemIndex, GetMemMask, SetBit, ClrBit, GetBit, InsElem, DelElem и IsMember. Они выполняют ограниченное число операций над одним машинным словом.

К O(N) относятся конструктор и копирование TBitField, присваивание, сравнение, побитовые операции, ввод и вывод. Эти алгоритмы обрабатывают массив слов или все логические позиции.

Для TSet линейную сложность имеют конструкторы, копирование, присваивание, сравнение, объединение, пересечение, дополнение, ввод и вывод. Операции над одним элементом имеют O(1), а операторы + и - с элементом имеют O(N), поскольку создают копию.

Память одного TBitField составляет ceil(N/W) элементов TELEM, то есть O(N) битов. TSet хранит один характеристический бит на каждый элемент универсума, поэтому также требует O(N) памяти.

Хотя асимптотическая запись не отражает константы, битовое представление компактно: одна машинная операция может одновременно обработать W битов.

4. Результаты экспериментов и тестирования

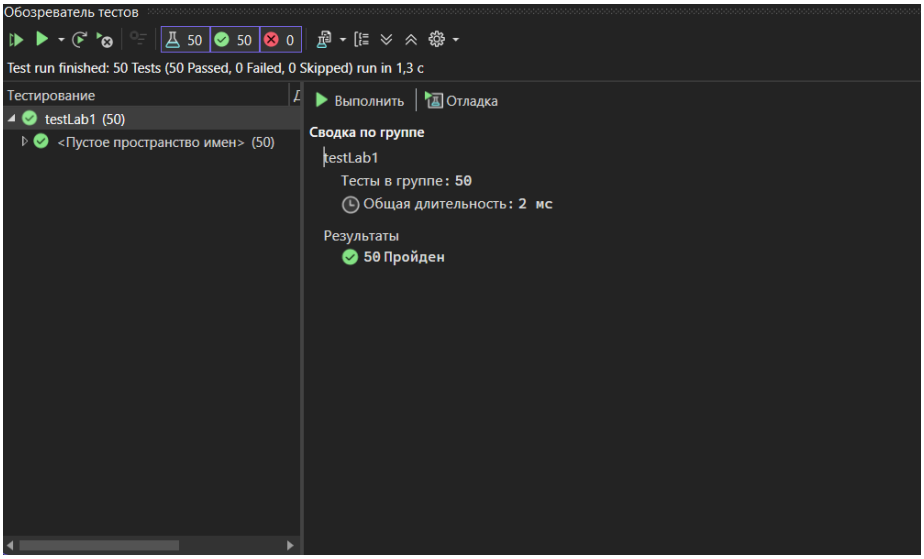
Для проверки использован Google Test. Тесты разделены на группы TBitFieldTest и TSetTest. Они создают объекты, выполняют операции и сравнивают фактические результаты с ожидаемыми через EXPECT_EQ и другие проверки.

Для TBitField проверяются длина, установка и чтение бита, очистка, недопустимые индексы, побитовые ИЛИ, И и дополнение, сравнение, присваивание полей разных размеров, несовпадающие размеры операндов и некорректный ввод.

Для TSet проверяются добавление и удаление элемента, принадлежность, недопустимые номера, объединение, пересечение, дополнение, сравнение, отрицательная мощность, присваивание множеств разных мощностей, несовпадающие универсумы и некорректный ввод.

Тесты разных размеров подтверждают, что после присваивания объект получает длину и содержимое источника. Тесты несовпадающих размеров подтверждают, что побитовые операции возвращают пустое поле, а операции над множествами — пустое множество.

Тесты некорректного ввода проверяют установку failbit при значении, отличном от 0 и 1, а также то, что обработанные до ошибки позиции уже изменены. По результатам запусков в Visual Studio, сообщённым в ходе разработки, тесты проходили успешно.



4.1. Анализ покрытия тестами

Проверки доступа к битам охватывают установку, чтение и очистку. Тесты границ проверяют отрицательный индекс и индекс, равный длине поля. Это позволяет убедиться, что методы не обращаются за пределы выделенного массива.

Побитовые операции проверяются на небольших заранее заданных наборах битов, для которых результат можно вычислить вручную. Проверяется каждый разряд результата, а не только факт создания объекта.

Для TSet проверяются включение и удаление элементов, принадлежность, а также операции над множествами. Тесты краевых случаев охватывают отрицательную мощность, несовпадающие размеры и присваивание объектов разных размеров.

Потоковый ввод проверяется на корректных значениях и на числе, отличном от 0 и 1. При ошибочном значении ожидается установка failbit и прекращение обработки последующих позиций.

Набор тестов таким образом проверяет функциональность обоих классов и важные граничные условия. Для сдачи рекомендуется добавить скриншот фактического успешного запуска тестов в Visual Studio Test Explorer.

Заключение

В ходе лабораторной работы реализованы классы TBitField и TSet. TBitField хранит последовательность битов в динамическом массиве TELEM, а TSet использует битовое поле в качестве характеристического вектора конечного множества.

Реализованы доступ к битам и элементам, копирование, присваивание, сравнение, побитовые операции, объединение, пересечение, дополнение, потоковый ввод и вывод.

Операции над одним битом или элементом выполняются за O(1). Операции над всем полем или множеством имеют сложность O(N). Память для хранения множества составляет O(N) битов.

Созданы автоматические тесты Google Test для проверки основного функционала и граничных случаев. В ходе разработки обработаны отрицательные размеры, некорректные индексы, несовпадающие размеры операндов и ошибки ввода.

Поставленная цель достигнута: разработана реализация конечного множества на основе битового поля и проверена её работоспособность.

Литература

1. Страуструп Бьерн. Язык программирования C++. Специальное издание. Пер. с англ. — М.: Издательство Бином, 2017 г. — 1136 с.

2. Сайт cppreference: https://en.cppreference.com/

Приложение А. Формулы и основные операции

Количество элементов массива: MemLen = (BitLen + bitsElement − 1) / bitsElement, где bitsElement = sizeof(TELEM) * 8.

Индекс слова для бита n: GetMemIndex(n) = n / bitsElement.

Маска для бита n: GetMemMask(n) = TELEM(1) << (n % bitsElement).

Установка: pMem[index] |= mask. Очистка: pMem[index] &= ~mask. Чтение: (pMem[index] & mask) != 0.

Объединение множеств соответствует побитовому ИЛИ характеристических векторов. Пересечение соответствует побитовому И. Дополнение инвертирует каждый допустимый бит относительно универсума.

При несовпадении длин битовых полей операции | и & возвращают TBitField(0). При несовпадении MaxPower объединение и пересечение множеств возвращают TSet(0).

Приложение Б. Сведения о тестировании

Тесты написаны с использованием Google Test. В тестовом файле применяются макросы TEST, EXPECT_EQ и EXPECT_TRUE. Каждый тест создаёт объект, выполняет операции и проверяет ожидаемое состояние.

Проверяемые сценарии включают установку и очистку бита, чтение за границами поля, побитовые операции, сравнение, добавление и удаление элемента множества, проверку принадлежности, объединение, пересечение, дополнение, присваивание разных размеров и ошибочный ввод.

В тестах используются небольшие размеры полей и множеств, поэтому ожидаемые результаты можно проверить вручную. Это упрощает обнаружение и локализацию ошибок.

При окончательном оформлении отчёта можно добавить снимок окна Test Explorer с результатом последнего запуска тестов. Снимок должен быть сделан после фактической сборки и запуска проекта.